

COINCRAFT RESEARCH

스마트 컨트랙트 개요

Chapter 1 · 1-1

2026 | CoinCraft Academy

COINCRAFT
ACADEMY

01

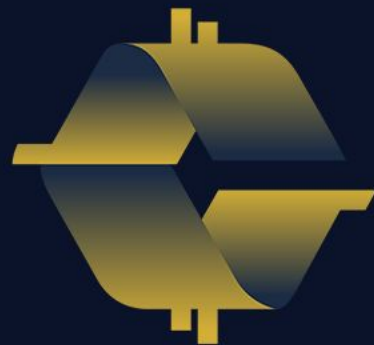
스마트 컨트랙트란 무엇인가

개념의 기원 · 닉 재보(1994) · 자판기 비유
코드가 법이다

Nick Szabo

1994

자판기 비유



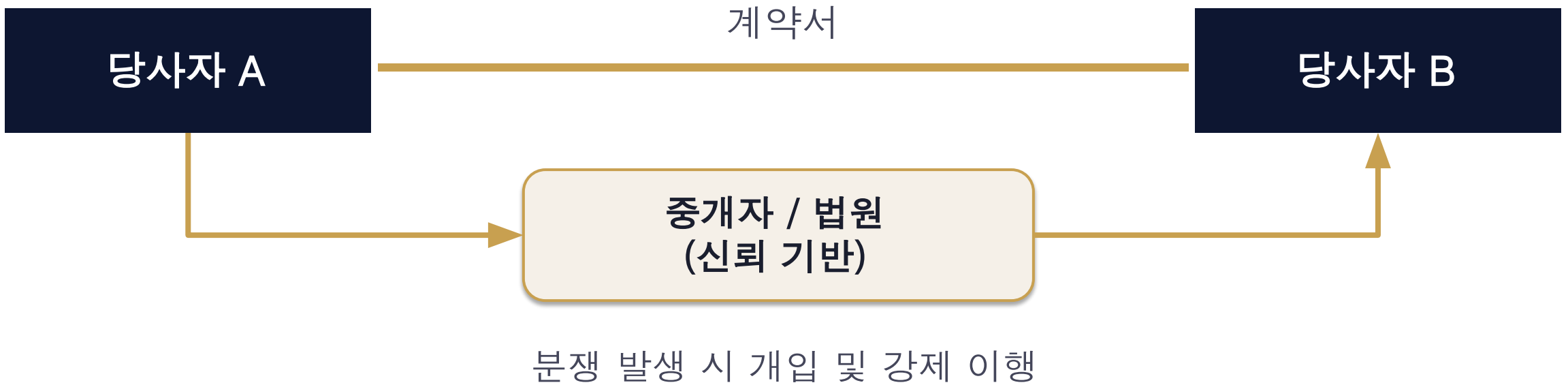
COINCRAFT
ACADEMY

계약이란 무엇인가

- 1 계약 = 조건과 이행 — "내가 돈을 주면, 너는 물건을 넘긴다"
- 2 전통 계약엔 계약서 + 중개자 + 법원이 반드시 필요
- 3 핵심 약점: 신뢰 — 상대방, 중개자, 법원 모두를 믿어야 한다
- 4 신뢰가 깨지면 계약도 무너진다

계약의 본질은 '조건과 이행'. 문제는 항상 '누구를 믿느냐'에 있다.

전통적 계약의 구조



전통 계약은 반드시 신뢰할 수 있는 제3자(중개자/법원)를 필요로 한다.

코드로 만든 계약 — 닉 재보(1994)

- 1 1994년, 암호학자 닉 재보(Nick Szabo): 계약을 코드로 작성하면?
- 2 코드는 감정 없고, 뇌물 없으며, 조건 충족 시 반드시 실행
- 3 신뢰할 수 있는 제3자 없이도 계약 이행 가능
- 4 이름: 스마트컨트랙트(Smart Contract)
- 5 비유: 자판기 — 기계에 내장된 규칙이 스스로 집행



아이디어는 천재적이었으나, '신뢰 없는 서버'가 없어 21년 동안 구현 불가능했다.

자판기 — 스마트컨트랙트의 원형



자판기 안에 내장된 규칙

- 1,000원 투입 → 음료 1개 배출
- 잔액 부족 시 동작 안 함
- 자판기 주인을 신뢰할 필요 없음 — 기계 자체를 신뢰

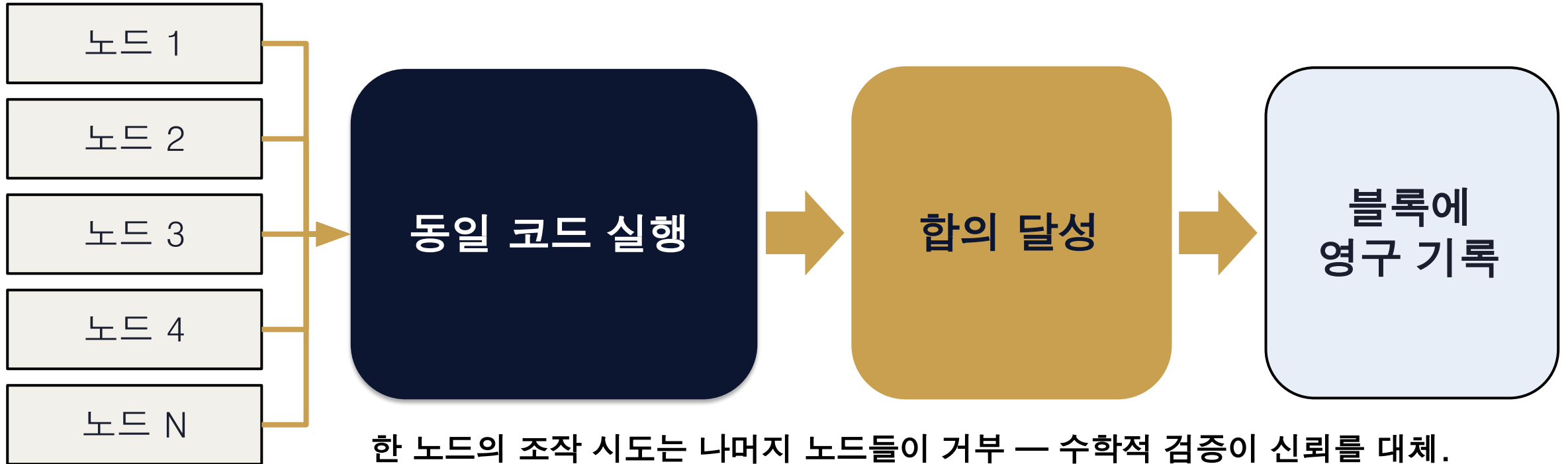
자판기 = 규칙이 기계 안에 내장된 계약. 스마트컨트랙트는 이것의 블록체인 버전.

블록체인 위에서 실행되는 코드

- 1 2015년 이더리움 등장 — 코드를 실행할 수 있는 분산 컴퓨터
- 2 전 세계 수천 개 노드가 동일한 코드를 동시에 실행 + 결과 검증
- 3 어느 한 노드가 조작해도 나머지가 동의하지 않으면 무효
- 4 블록체인에 배포된 코드 = 스마트컨트랙트

스마트컨트랙트:
블록체인 네트워크에 배포되어, 미리 정해진 조건이 충족되면 자동으로 실행되는 프로그램

스마트컨트랙트 실행 환경



코드가 "법"이고 "판사"이며 "집행관"이다

21년의 공백

1994	닉 재보, 스마트컨트랙트 개념 최초 제안
2008	나카모토 사토시, 비트코인 백서 — 분산 합의 메커니즘 첫 구현
2009	비트코인 네트워크 가동 — 중앙기관 없는 가치 이전 증명
2013	비탈릭 부테린(19세), 이더리움 백서 — 튜링 완전 언어 추가 제안
2015	이더리움 메인넷 출범 — 닉 재보의 아이디어 21년 만에 현실화

블록체인(2008) + 분산 합의 + 튜링 완전 언어 — 세 조각이 합쳐진 후에야 가능했다.

02

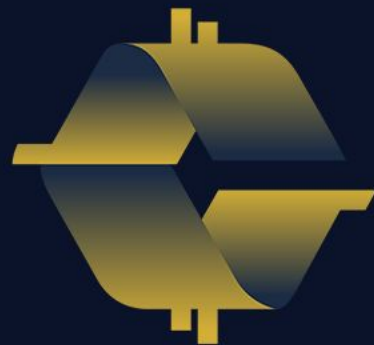
기존 서버 프로그램과의 차이

불변성 · 투명성 · 검열 불가

배포 후 수정 불가 — The DAO 사례

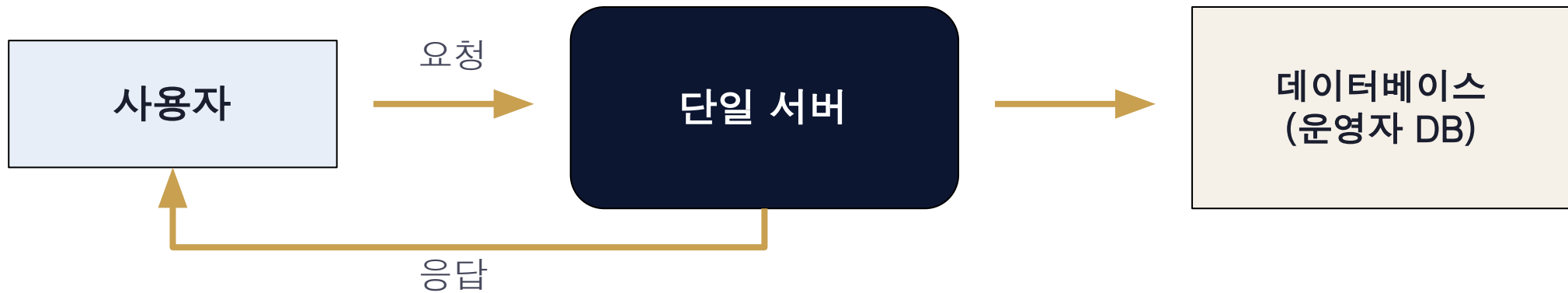
중앙화 vs 탈중앙화

배포 후 수정 불가



COINCRAFT
ACADEMY

중앙화 서버 모델



중앙화 서버의 특성

- 모든 데이터는 서버 운영자의 DB에 존재
- 운영자가 언제든지 데이터 변경 가능
- 운영자가 서비스 종료 시 사라짐
- 신뢰 = 운영자를 믿는 것

카카오뱅크 송금: 내 잔액 변경은 카카오뱅크 DB에서 일어난다. 이것을 알아챌 수 없다.

중앙화 서버 vs 스마트컨트랙트

중앙화 서버

- 실행 주체: 특정 서버 (단일 운영자)
- 데이터 저장: 운영자 데이터베이스
- 신뢰 기반: 운영자
- 코드 투명성: 비공개 가능
- 수정 가능 여부: 운영자 마음대로
- 장점: 빠름, 저렴, 유연
- 단점: 단일 실패점, 검열 가능

스마트컨트랙트

- 실행 주체: 블록체인 네트워크 (탈중앙화)
- 데이터 저장: 블록체인 (모두에게 공개)
- 신뢰 기반: 코드 자체 (수학적 검증)
- 코드 투명성: 누구나 확인 가능
- 수정 가능 여부: 배포 후 변경 불가*
- 장점: 신뢰 불필요, 검열 저항
- 단점: 느림, 비쌈, 버그 수정 불가

* 업그레이드 패턴 사용 시 가능하지만, 의도적 설계가 필요.

차이 ① — 불변성(Immutability)

- 1 배포된 스마트컨트랙트는 수정할 수 없다
- 2 한번 블록체인에 올라간 코드는 영원히 그 상태로 존재
- 3 장점: 운영자가 규칙을 몰래 바꿀 수 없음
- 4 단점: 버그를 발견해도 수정 불가

The DAO 해킹 사건 (2016) — 불변성의 양날의 검

재진입 버그 → 6,000만 달러 탈취 → 코드 수정 불가 → 이더리움 하드포크 결정
→ 이더리움(ETH) vs 이더리움 클래식(ETC) 분리 — 코드 수정 불가가 네트워크 분열을 낳았다

"코드가 법이다" — 잘못된 코드는 영구적으로 잘못된 채로 남는다.

차이 ② — 투명성(Transparency)

- 1 배포된 컨트랙트의 코드와 모든 실행 결과는 블록체인에 공개
- 2 누구든 Etherscan에서 코드와 트랜잭션 기록을 확인 가능
- 3 "Code is Law" — 조건 충족 시 코드는 반드시 실행
- 4 운영자의 의도나 법원의 판결이 개입할 여지 없음

중앙화 서버
코드 비공개 가능
운영자만 알고 있음

스마트컨트랙트
누구나 코드 확인 가능
Etherscan에서 투명하게 공개

투명성은 신뢰의 기반 — 감사(Audit) 없이도 코드 자체로 검증 가능.

백엔드 개발자가 스마트컨트랙트로 전환할 때

가스 비용	DB 쓰기가 수만 원 — 데이터 1바이트도 비용 발생
수정 불가	배포 후 버그 발견 → 수정 불가, 재배포도 의미 없음
결정론적 환경	현재 시간·난수·외부 API 호출 불가 — 별도 설계 필요
금전 직결	ETH 전송 코드 한 줄이 재진입 공격 대상이 됨

패러다임 전환 — 코드가 돈을 직접 다루는 환경에서의 개발은 다르다.

스마트컨트랙트 모델 — 탈중앙화 실행



스마트컨트랙트 실행의 특성

- 코드는 블록체인에 영구 저장
- 수천 개 노드가 동일하게 실행
- 어느 한 노드의 조작 불가
- 신뢰 = 코드 자체 (수학적 검증)

신뢰할 서버 운영자가 없다 — 네트워크 자체가 신뢰의 기반.

03

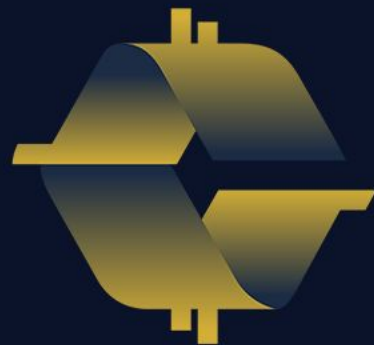
블록체인에서 코드가 실행되는 방식

트랜잭션 · 합의 · 결정론적 실행
전 세계 노드가 동일한 결과를 낸다

트랜잭션

합의

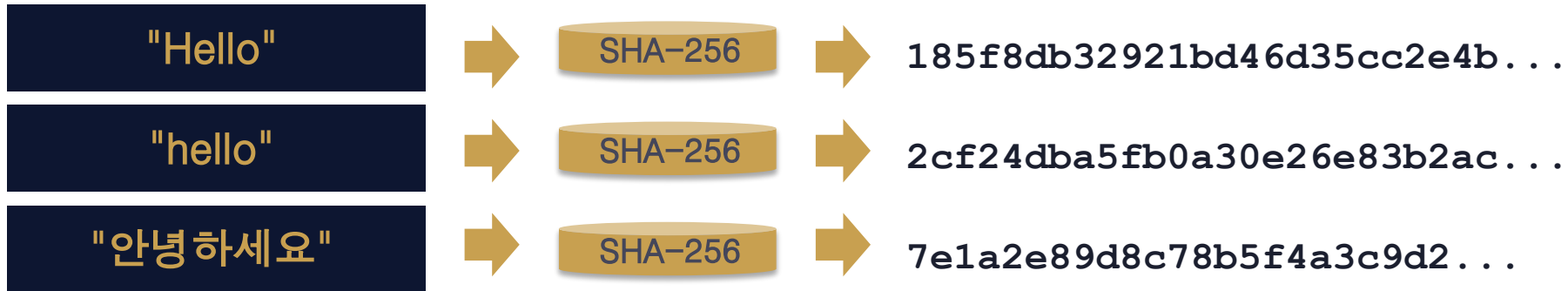
결정론적 실행



COINCRAFT
ACADEMY

해시(Hash) — 블록체인 불변성의 핵심

- 1 해시 함수: 임의의 데이터 → 고정 크기 문자열로 변환
- 2 단방향: 해시에서 원본 복원 불가
- 3 결정론적: 같은 입력 → 항상 같은 해시
- 4 눈사태 효과: 입력 1글자 변경 → 해시 완전히 달라짐



이 특성 덕분에 블록에 이전 블록 해시를 포함하면 변조를 즉시 탐지할 수 있다.

합의 알고리즘 — PoW에서 PoS로

작업증명 (PoW) — 2022년 이전 이더리움

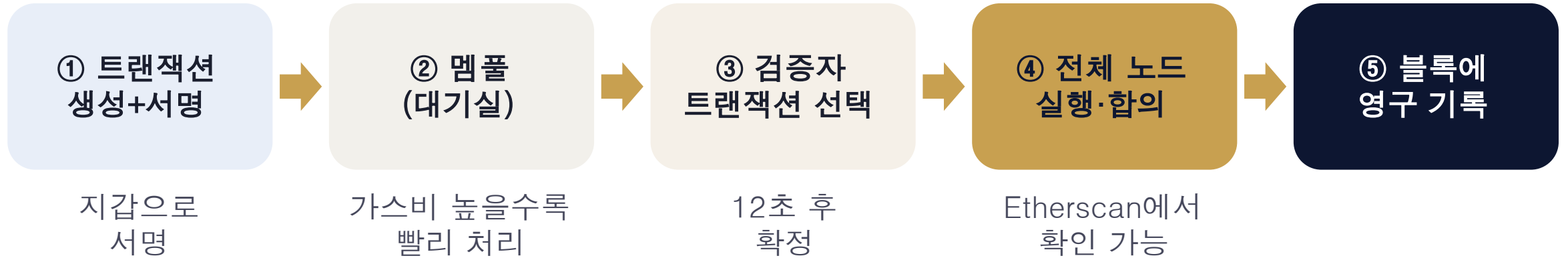
- 채굴자들이 복잡한 수학 문제를 먼저 풀면 블록 생성
- 해결책이 올바르면 다른 노드들이 검증 후 수용
- 전기와 컴퓨팅 파워가 '일'의 증명
- 에너지 소비 매우 높음
- 현재도 비트코인이 사용 중

지분증명 (PoS) — 현재 이더리움

- 32 ETH를 스테이킹한 검증자들이 블록 생성권 보유
- 무작위 선택된 검증자가 블록 제안, 나머지 투표
- 나쁜 행동 시 스테이킹 ETH 삭감(슬래싱)
- 에너지 소비 매우 낮음 (99% 이상 감소)
- 2022년 9월 이더리움이 PoW → PoS 전환 (The Merge)

이더리움 The Merge(2022.09): PoW → PoS 전환으로 에너지 소비 99% 감소.

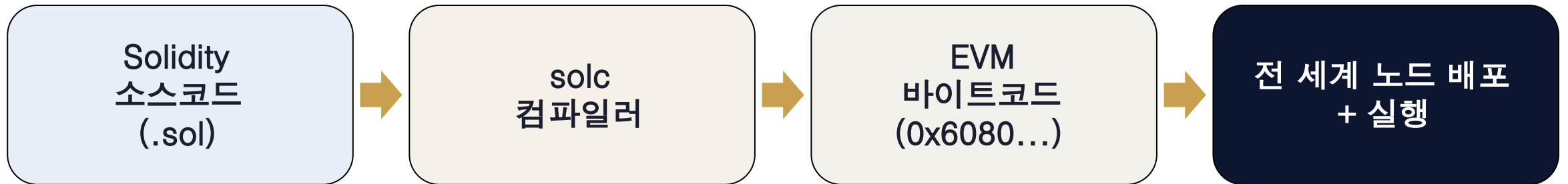
트랜잭션이 블록체인에 기록되는 과정



이더리움 기준 약 12초마다 블록 생성 — 그 사이 내 트랜잭션은 댐풀에서 대기.

EVM — 스마트컨트랙트의 실행 엔진

- 1 EVM(Ethereum Virtual Machine): 전 세계 모든 이더리움 노드에 탑재된 가상 머신
- 2 Solidity 코드는 컴파일러(solc)를 통해 EVM 바이트코드로 변환
- 3 전 세계 노드들이 동일한 바이트코드를 동일하게 실행
- 4 결정론적(Deterministic): 같은 입력 + 같은 상태 → 항상 같은 결과



결정론적 실행 — 운영체제/위치 무관, 항상 동일한 결과. 이것이 합의의 기반.

가스(Gas) — 실행 비용의 단위

- 1 EVM은 모든 연산(덧셈, 저장, 함수 호출 등)에 가스 비용을 매긴다
- 2 가스 한도 초과 시 실행 중단, 가스는 소진 — 돌이킬 수 없다
- 3 가스비가 없으면 무한 루프 코드로 네트워크 전체를 멈출 수 있다
- 4 이 구조가 이더리움 네트워크를 악의적 코드로부터 보호

가스 한도 설정
(Gas Limit)
트랜잭션 당 최대 비용
초과 시 Revert

가스 가격
(Gas Price)
높을수록 빨리 처리
시장 원리로 결정

가스비 = 한도 × 가격
스토리지 쓰기 비용 가장 높음
최적화가 곧 비용 절감

데이터 1바이트 더 저장할수록 비용 증가 — 스마트컨트랙트 설계 자체를 바꾼다.

04

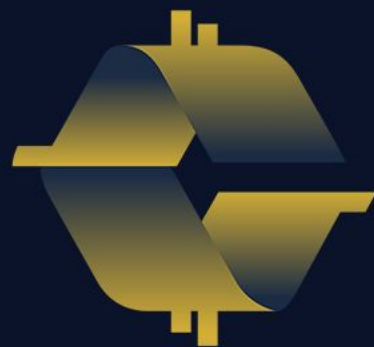
스마트 컨트랙트의 활용 사례

DeFi · NFT · RWA · 보험 자동화
교보생명 디지털 자산 발행

DeFi

NFT

RWA



COINCRAFT
ACADEMY

이미 수조 원이 스마트컨트랙트 위에서 운영 중

토큰 발행

ERC-20: USDC, USDT 등 스테이블코인

탈중앙 거래소

DEX: Uniswap — 서버 없는 거래소

DeFi 대출

Aave, Compound — 자동
담보·이자·청산

NFT

ERC-721/1155: 고유 디지털 자산
소유권

DAO 거버넌스

투표 → 자동 실행 — 코드가 조직 운영

RWA 토큰화

블록록 BUIDL: 국채 → 토큰, 초 단위 처리

DeFi TVL 2024년 기준 약 500억 달러 이상 — 스마트컨트랙트에 잠겨 있는 자산 총액.

05

이더리움 네트워크

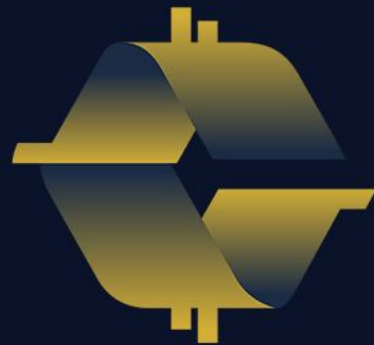
개인키 · 공개키 · 주소 도출

트랜잭션 구조 · Gas · EIP-1559

ECDSA

Keccak-256

EIP-1559



COINCRAFT
ACADEMY

스마트컨트랙트의 한계와 주의점

오라클 문제

블록체인은 외부 데이터 직접 접근 불가
→ Chainlink 등 오라클 필요, 오라클 오류 시 컨트랙트도 오류

가스 비용

이더리움 메인넷 혼잡 시 단순 전송도 수만 원
→ 레이어2가 완화 중이지만, 최적화는 여전히 필수

업그레이드 딜레마

불변성 = 신뢰 기반 + 버그 수정 불가
→ 프록시 패턴 사용 시 업그레이드 가능, 그러나 중앙화 위험 발생

"가비지 인, 가비지 아웃" — 입력 데이터(오라클)가 잘못되면 완벽한 코드도 오류.

학습 필수 도구와 실전 팁

Etherscan.io	이더리움 블록 탐색기 — 실제 트랜잭션·컨트랙트 확인 최고의 학습 도구
Remix IDE	remix.ethereum.org — 설치 없이 브라우저에서 Solidity 작성·배포 가능
MetaMask	이더리움 지갑 — 테스트넷에서 실제 트랜잭션 경험 필수 도구
Sepolia Testnet	실제 ETH 없이 테스트 가능한 공개 테스트 네트워크

주의사항

- 테스트넷과 메인넷을 구분하라 (테스트넷 ETH는 가치 없음)
- 프라이빗 키를 절대 공개하지 마라 — 깃허브에 올리면 봇이 즉시 탈취
- 블록체인에 올라간 데이터는 수정 불가 — 개인정보를 온체인에 올리지 마라

Remix + Sepolia로 시작하라 — 실제 배포 경험이 가장 빠른 학습 경로.

Remix vs Hardhat

DEVELOPMENT ENVIRONMENT COMPARISON

항목	Remix IDE	Hardhat
설치	없음 — 브라우저에서 바로 실행	Node.js + npm install 필요
실행 환경	브라우저 내 JS EVM (ethereumjs-vm)	Node.js Hardhat Network
테스트	수동 클릭 — 버튼 누르며 확인	코드 작성 (Mocha / Chai)
협업 / git	개인 브라우저 — 공유 불편	git 기반 프로젝트 구조
CI/CD	불가	가능 (GitHub Actions 등)
mainnet fork	불가	가능 — 실제 메인넷 상태 로컬 복제
console.log	불가	Solidity 안에서 가능
이 과정	S33 ~ S39 기초 실습 개념 확인, 즉석 실험	S40 ~ S42 KyoboNFT 구현 실제 Phase 1 코드

S33~S39는 Remix로 개념을 빠르게 확인한다. S40부터 Hardhat으로 전환해 실제 KyoboNFT를 배포·테스트한다.

06

EOA와 CA

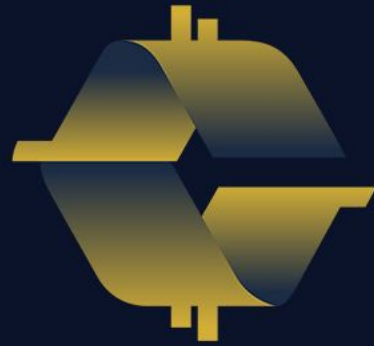
외부 소유 계정 · 컨트랙트 계정

모든 트랜잭션의 시작점은 반드시 EOA

EOA

Contract Account

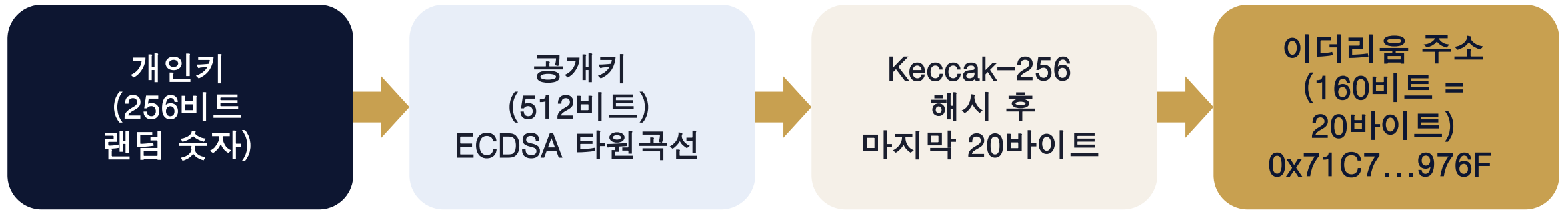
개인키 유무



COINCRAFT
ACADEMY

개인키 → 공개키 → 이더리움 주소

0xac0974bec39a17e36ba4a6b4d238ff944bacb478...



단방향 (역산 불가): 주소 → 공개키 → 개인키 복원 수학적으로 불가능

주소를 알아도 개인키를 역산할 수 없다 — 256비트 공간의 충돌 확률은 사실상 0.

EOA vs Contract Account 비교

EOA (외부 소유 계정)

- 생성: 개인키 생성
- 제어: 개인키 서명
- 코드: 없음
- 트랜잭션 시작: 가능 (능동)
- ETH 보유: 가능
- 상태 저장: 잔액만
- 개인키: 있음

Contract Account (컨트랙트 계정)

- 생성: 스마트컨트랙트 배포
- 제어: 코드 로직
- 코드: 있음 (불변 바이트코드)
- 트랜잭션 시작: 불가 (수동, 호출받아야 함)
- ETH 보유: 가능
- 상태 저장: 변수들 (Storage)
- 개인키: 없음

모든 트랜잭션의 시작점은 반드시 EOA다 — 컨트랙트는 EOA의 호출에 의해서만 실행.

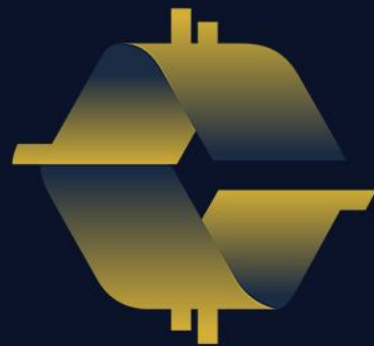
07 EVM

스택 기반 가상 머신 · Opcode · Gas 미터링
Storage · Memory · Stack · Calldata

Stack 기반

Opcode

Gas 미터링



COINCRAFT
ACADEMY

가상 머신(VM)이란 무엇인가

- 1 가상 머신 = 하드웨어에 독립적인 소프트웨어 실행 환경
- 2 JVM: Java 코드 → 바이트코드 → 어느 OS에서든 동일하게 실행
- 3 EVM: Solidity → EVM 바이트코드 → 전 세계 노드에서 동일하게 실행
- 4 핵심 공통점: '한 번 작성, 어디서나 실행' — 결정론적 실행 보장
- 5 차이점: EVM은 가스 비용·블록체인 상태 접근·비결정론적 작업 금지

EVM = 이더리움의 모든 노드에 탑재된 표준 실행 환경. 어디서든 같은 결과.

EVM의 3가지 핵심 특성

격리 (Isolation)

각 컨트랙트는 독립된 샌드박스
다른 컨트랙트 상태에 직접
접근 불가
명시적 함수 호출로만 상호작용

결정론 (Determinism)

동일 입력 + 동일 상태
→ 항상 동일한 결과
수천 노드 합의의 필수 조건

가스 미터링 (Gas Metering)

모든 연산(Opcode)에 비용
부과
가스 한도 초과 시 실행 중단
무한 루프·스팸 방지

결정론이 없으면 노드마다 다른 결과 → 합의 불가 → 블록체인 붕괴.

EVM = 스택 기반 머신

1 스택(Stack) = 후입선출(LIFO) 구조의 데이터 공간

2 최대 1024개의 32바이트(256비트) 값 저장

3 연산은 스택 맨 위에서부터 필요한 만큼 꺼내 사용

```
Solidity: uint256 result = 3 + 5;
```

PUSH1 0x03

스택: [3]

PUSH1 0x05

스택: [5, 3]

ADD

스택: [8] (5+3)

Opcode = EVM의 최소 명령 단위

Solidity 코드 → 컴파일러(solc) → EVM 바이트코드(Opcode 연속)

예: ADD(덧셈·3 gas), SSTORE(스토리지 쓰기·20,000 gas, 첫쓰기 기준), REVERT(취소)

바이트코드 = 16진수 문자열. EVM이 한 Opcode씩 읽으며 실행.

Solidity 개발자가 모든 Opcode를 외울 필요는 없다. 하지만 가스 차이를 알아야 한다.

주요 EVM Opcode와 가스 비용

ADD / MUL	덧셈 / 곱셈 3 / 5 gas	SSTORE	스토리지에 쓰기 2900~20000 gas (첫 쓰기 기준)
SHA3	Keccak-256 해시 30+ gas	CREATE	새 컨트랙트 생성 32000 gas
CALLER	호출자 주소 push 2 gas	CALL	다른 계정/컨트랙트 호출 700+ gas
SLOAD	스토리지에서 읽기 100~2100 gas	REVERT	실행 취소, 가스 반환 가스 반환

SSTORE(20,000 gas) = ADD(3 gas)의 6,666배. 상태 변수 최소화가 가스 최적화의 핵심.

EVM 데이터 저장 공간 3가지

Storage (스토리지)	Memory (메모리)	Calldata	Stack (스택)
영구 저장 블록체인에 기록 SSTORE: 20,000 gas 상태 변수 보관	임시 저장 함수 실행 중만 존재 중간 비용 함수 내 임시 데이터	읽기 전용 입력 트랜잭션 입력 데이터 가장 저렴 external 파라미터	EVM 연산 작업공간 1024 × 32바이트 가장 저렴·빠름 직접 제어 불가

```
// Storage (영구):  uint256 public totalSupply;    // 슬롯 0 - 비쌈
// Memory  (임시):  uint256 result = a + b;         // 함수 내 임시값
```

Storage → Memory → Calldata → Stack 순으로 저렴. 저장 위치가 가스비를 결정한다.

Solidity → EVM 실행 흐름



ABI (Application Binary Interface)

컨트랙트 함수의 이름·파라미터 타입·반환값을 JSON으로 정의한 인터페이스 명세서
ethers.js·web3.js 등 라이브러리가 ABI를 기반으로 함수 호출을 자동 인코딩·디코딩

ABI = 컨트랙트와 외부 세계의 계약서. 함수 선택자 = 함수 서명 해시의 앞 4바이트.

ABI — 함수 선택자 계산

FUNCTION SELECTOR — KECCAK256 FIRST 4 BYTES

함수 선택자 계산 원리

```
keccak256("mint(address,uint256,uint256,bytes)")  
= 0x731133e9ebb09be3f374bc25dce3e2dd...
```

앞 4바이트만 → 0x731133e9 ← 함수 선택자

calldata 구조:

[0x731133e9] [파라미터 인코딩...]

4바이트 나머지

왜 4바이트? — $2^{32} = 42$ 억 개 선택자 공간. 충돌 확률 ↓ 컨
트랙트 함수 수는 수십 개 수준.

컴파일러가 충돌을 막아준다.

ethers.js로 직접 계산

```
import { ethers } from "ethers";  
  
// mint(address,uint256,uint256,bytes)  
const selector = ethers.id(  
  "mint(address,uint256,uint256,bytes)"  
)  
.slice(0, 10);  
console.log(selector); // "0x731133e9"  
  
// ERC-20 transfer  
const transfer = ethers.id(  
  "transfer(address,uint256)"  
)  
.slice(0, 10);  
console.log(transfer); // "0xa9059cbb"
```

fallback — 4바이트가 어떤 함수와도 매칭 안 되면 `fallback()` 실행. ABI 없이 raw calldata를 보내면 fallback 호출.

Solidity — Hello.sol

FIRST CONTRACT — FULL CODE WITH ANNOTATIONS

Hello.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
contract Hello {
    string public greeting = "Hello"; // storage - 영구
    constructor() {
        greeting = "Hello, Kyobo!"; // 배포 시 1회
    }
    // view: gas 없음, 즉시 반환
    function greet() public view returns (string memory) {
        return greeting;    트랜잭션 내부 호출 시에는 연산 gas 발생.
    }
    event Greeted(address indexed who, string message);
    function sayHello() public {
        emit Greeted(msg.sender, greeting); // 로그 기록
    }
    function setGreeting(string memory newGreeting) public {
        greeting = newGreeting; // 트랜잭션 필요
    }
}
```

핵심 포인트

- `string public greeting` → 상태변수 = Storage (영구, SSTORE 비쌈)
- `constructor()` → 배포 시 1회만 실행. 이후 호출 불가
- `view` → 트랜잭션 불필요. gas 0. 즉시 반환
- `string memory` → 참조 타입은 위치 명시 필수 (값 타입은 불필요)
- `emit Greeted(...)` → 트랜잭션 로그 기록. Storage 아님 → 저렴
- `msg.sender` → 현재 호출자 주소 (전역변수)
- `setGreeting()` → 상태 변경 → 트랜잭션 필요 → gas 소비

공개 setter 주의 — 현재 `setGreeting` 은 누구나 호출 가능. 실제 컨트랙트는 `onlyOwner` 추가 필요.

Solidity 핵심 키워드

SOLIDITY KEYWORDS — VISIBILITY · MUTABILITY · DATA LOCATION

키워드	위치	의미	gas / 트랜잭션
public	함수·변수	누구나 호출 가능. 변수는 자동 getter 생성	—
view	함수	상태 읽기만. 변경 없음	gas 없음
pure	함수	상태 접근 없음. 순수 계산만	gas 없음
payable	함수	ETH를 수신할 수 있음	트랜잭션
memory	타입 위치	함수 실행 중 임시 저장. 종료 시 소멸	중간
storage	타입 위치	블록체인 영구 저장. SSTORE 비용	높음
calldata	타입 위치	읽기 전용 입력. 가장 저렴	최저
emit	구문	이벤트 발행 → 트랜잭션 로그에 기록	낮음
msg.sender	전역변수	현재 함수를 호출한 주소 (EOA 또는 컨트랙트)	—
msg.value	전역변수	함께 전송된 ETH 양 (wei 단위)	—

위치 키워드 — memory / storage / calldata 는 참조 타입(string, bytes, 배열, struct)에만 명시. uint256·address 같은 값 타입은 불필요.

08

상태 · 트랜잭션 · 이벤트

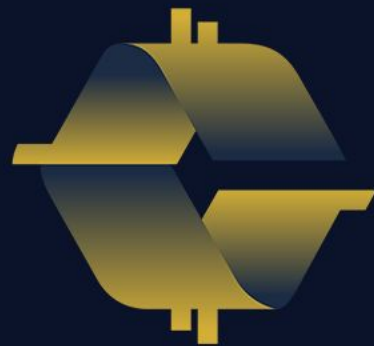
상태 전이 · view vs 상태 변경 함수

이벤트 로그 · indexed 필터링

SSTORE

emit

indexed



COINCRAFT
ACADEMY

이더리움 = 거대한 상태 기계

- 1 이더리움 전체 = 수백만 계정의 잔액·코드·저장소 스냅샷
- 2 트랜잭션이 입력값 → EVM이 상태를 새 값으로 전이
- 3 블록마다 전 세계 노드가 동일한 '다음 상태'에 합의
- 4 상태 루트(State Root): Merkle Patricia Trie로 압축 저장

이더리움 = 트랜잭션으로 구동되는 분산 상태 기계

상태 전이 구조



같은 트랜잭션을 실행하면 어느 노드든 반드시 같은 S(n+1)을 얻는다

결정론적(Deterministic) 실행 — 입력이 같으면 출력이 반드시 같다

트랜잭션의 구조

to	수신 주소 (0x0 = 컨트랙트 배포)
value	전송할 ETH 양 (wei 단위)
data	함수 호출 ABI 인코딩 (EOA 간 전송 시 비어 있음)
gasLimit	사용 허용 최대 가스
maxFeePerGas	가스 1단위당 최대 지불 가격 (EIP-1559)
nonce	계정이 발행한 트랜잭션 일련번호
v, r, s	ECDSA 서명 — 발신자 신원 증명

nonce는 재전송 공격을 막는 일련번호 — 빠진 nonce가 있으면 트랜잭션이 대기한다

읽기 vs 쓰기 — view와 상태 변경 함수

view / pure 함수

- 트랜잭션 불필요
- 가스비 0 (로컬 실행)
- 블록체인 상태 변경 없음
- 즉시 결과 반환
- 예: `balanceOf()`, `name()`

상태 변경 함수

- 트랜잭션 필요 (서명)
- 가스비 발생
- 블록체인에 영구 기록
- 채굴 대기 시간 존재
- 예: `transfer()`, `approve()`

읽기는 무료·즉시 / 쓰기는 비용·대기 — 설계의 핵심 구분

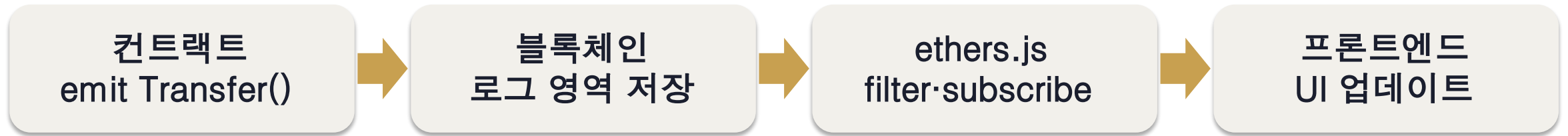
이벤트(Event) — 블록체인의 로그

- 1 이벤트 = 트랜잭션 실행 중 방출하는 로그 메시지
- 2 블록체인 상태(Storage)가 아닌 로그 영역에 저장 → 가스 절약
- 3 indexed 파라미터: 빠른 검색 가능 (최대 3개)
- 4 프론트엔드·백엔드가 ethers.js로 이벤트를 구독·필터링
- 5 배포 이후 이벤트가 유일한 히스토리 추적 수단

emit Transfer(from, to, amount) → Etherscan '토큰 이동 탭'에 표시

Storage에 쓰면 비싸다. 이벤트로 남기면 가스가 3~10배 저렴하다.

이벤트 발행 → 구독 흐름



이벤트 없이는 온체인 히스토리를 효율적으로 조회할 수 없다

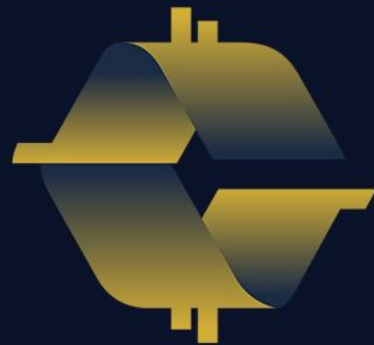
실습 Remix 실습 3가지 실험

Hello.sol · GasTest.sol · 함수 선택자 계산

Remix VM

10분

브라우저 실행



COINCRAFT
ACADEMY

실습 1 — Hello.sol 배포

PRACTICE 1 · REMIX IDE · VIEW VS TRANSACTION

STEP BY STEP

- 1 **Remix 접속** — `remix.ethereum.org` → 좌측 파일 탐색기 → `contracts/` → 새 파일 `Hello.sol`
- 2 **컴파일** — Solidity Compiler 탭 → 버전 `0.8.20` 선택 → Compile → 초록 체크 확인
- 3 **배포** — Deploy & Run 탭 → Environment: `Remix VM (Cancun)` → Deploy 클릭
- 4 **view 확인** — `greet()` 클릭 → 하단 로그: `[call]` → gas 없음, 즉시 반환
- 5 **트랜잭션 확인** — `sayHello()` 클릭 → `[vm]` gas used: ~27,000
- 6 **상태 변경 확인** — `setGreeting("안녕, 교보!")` → `greet()` 재호출 → "안녕, 교보!" 반환

예상 결과

함수	로그 타입	gas
<code>greet()</code>	<code>call</code>	0
<code>sayHello()</code>	<code>vm</code>	~27,000
<code>setGreeting()</code>	<code>vm</code>	~34,000

ABI 확인 — Solidity Compiler 탭 → Compilation Details → ABI 탭 → JSON 구조 직접 확인. M7에서 ethers.js가 이걸 그대로 사용한다.

로그 타입 차이 — `[call]`은 트랜잭션 없이 로컬 읽기, `[vm]`은 블록에 기록되는 실제 트랜잭션.

실습 2 — GasTest.sol

PRACTICE 2 · STORAGE VS MEMORY GAS COMPARISON

GasTest.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
contract GasTest {
    uint256 public storedValue;

    // SSTORE: 20,000 gas (새 슬롯) / 2,900 gas (업데이트)
    function writeStorage(uint256 value) public {
        storedValue = value;
    }

    // pure: SSTORE 없음 — 스택·메모리만 사용
    function computeOnly(uint256 a, uint256 b)
        public pure returns (uint256) {
        return a + b;
    }
}
```

배포 후 각 함수 호출 → gas 비교

함수	gas used	이유
writeStorage(42)	~43,000	SSTORE 20,000 포함
computeOnly(3,5)	~21,400	기본 21,000만
writeStorage(42) ×2	~26,000	기존 슬롯 업데이트: 2,900

SSTORE = ADD의 6,666배 — 상태변수 최소화가 가스 최적화의 핵심.

심화 — `uint256 val = storedValue;` 로 먼저 캐싱하면 SLOAD가 2100→100으로 줄어든다.

실습 3 — 함수 선택자 직접 계산

PRACTICE 3 · FUNCTION SELECTOR IN REMIX CONSOLE

Remix 하단 터미널(콘솔)에서 실행

```
// greet() 선택자
const sig1 = "greet()";
const hash1 = web3.utils.keccak256(sig1);
console.log("greet() selector:", hash1.slice(0, 10));
// → "0xcfae3217"

// mint(address,uint256,uint256,bytes) 선택자
const sig2 = "mint(address,uint256,uint256,bytes)";
const hash2 = web3.utils.keccak256(sig2);
console.log("mint() selector:", hash2.slice(0, 10));
// → "0x731133e9"

// setGreeting(string) 선택자 - 직접 계산해보기
const sig3 = "setGreeting(string)";
console.log(web3.utils.keccak256(sig3).slice(0, 10));
```

확인 포인트

- 계산한 선택자와 Remix 배포 후 실제 calldata 첫 4바이트 비교
- Remix Deploy 탭 → 함수 호출 → 하단 로그 → `input` 필드 확인
- `input: 0xcfae3217` → `greet()` 호출임을 확인

이더스캔 연결 — 실제 트랜잭션의 Input Data 탭에서도 같은 4바이트를 확인할 수 있다. 선택자만 보고 어떤 함수인지 역추적 가능.

완료 기준 — `greet()` 선택자 `0xcfae3217`, `mint()` 선택자 `0x731133e9` 직접 계산 성공.

주의 — 파라미터 타입 철자가 다르면 선택자가 달라진다. `uint`이 아닌 `uint256` 을 써야 한다.